

Concurrencia en NodeJS

UD 4. Programación concurrente con NodeJS

Programació de Serveis
i Processos

CFGS Desenvolupament
d'Aplicacions Multiplataforma



IES Jaume II El Just
Tavernes de la Valldigna

Departament d'Informàtica
Curs 2022-23

Contenidos

1	Concurrencia en nodejs	3
1.1	El event Loop o bucle de eventos	4
2	Promesas en ES6	5
2.1	Ejemplo	5
2.2	Ventajas con el uso de promesas	8
2.2.1	Garantías	8
2.2.2	Encadenamiento	8
2.3	Ejemplo	8
2.3.1	async/await	11

1 Concurrencia en nodejs

JavaScript es un lenguaje **asíncrono y concurrente**. ¿Qué significan estos dos conceptos?

- **Asíncrono:** La programación tradicional se basa en el paradigma de *programación secuencial*, donde un programa es una secuencia de instrucciones que se ejecutan ordenadamente, y una operación no empieza hasta que no acaba el anterior. Se trata pues de un modelo síncrono. Frente a esto, la *programación asíncrona* da la posibilidad que algunas operaciones devuelvan el control de la ejecución al programa principal antes de haber finalizado, lo que agiliza el proceso de ejecución.
- **Concurrente:** La *conurrencia* se daba cuando sólo una tarea se estaba ejecutando en un momento dado, frente al paralelismo, donde había varias tareas ejecutándose al mismo tiempo.

Entonces, el hecho que javascript sea asíncrono y concurrente significa que las instrucciones no tienen por qué seguir un orden secuencial, de forma que no hay que esperar que acaben ciertas instrucciones para iniciar la ejecución otras, y que además, solo se está ejecutando una instrucción en un momento dado.

JavaScript (y por tanto nodejs) hacen uso de lo que se conoce como *event loop* o *bucle de eventos*, que es el componente encargado de coordinar la ejecución, los events y los callbacks. Vemos algunos de estos conceptos con un ejemplo:

```
function saluda(nombre) {  
  console.log("Hola " + nombre);  
}  
  
for (let i = 0; i < 5; i++) {  
  setTimeout(function() {  
    saluda(i);  
  }, 500 - i * 100);  
}  
  
process.nextTick(function() { saluda("Prioritario 1") });  
process.nextTick(function() { saluda("Prioritario 2") });  
  
console.log("Fin de la ejecución");
```

Analizamos el ejemplo: * Definimos una función *saluda*, que recibe un parámetro *nombre*, y escribe en la terminal "Ho la", seguido del nombre que le pasamos. * Posteriormente, lanzamos un bucle que cuenta de 0 a 5, y dentro, define un `setTimeout`. La función `setTimeout` lo que hace es poner en marcha un temporizador, que lanza una función al vencer este:

```
setTimeout(funcion, tiempo_en_ms);
```

- Como vemos, la función que se ejecuta al vencer el temporizador (se trata de una función anónima, puesto que no tiene nombre), lo que hace es invocar a la función *saluda*, pasándole como parámetro el valor *i* de control del bucle. Estas funciones que se lanzan como respuesta a un evento (como es el caso del vencimiento del temporizador) se denominan funciones de retorno o de *callback*.
- Además, al segundo parámetro de la función `setTimeout` le pasamos el tiempo que tardará al vencer este. En el ejemplo, hemos indicado que este tiempo sea mayor cuanto más pequeño sea el valor de *i*, según la fórmula $500 - i * 100$.
- Se han añadido al final dos instrucciones `process.nextTick`, que añaden dos funciones también a ejecutar. Después profundizaremos más en estas funciones.
- Y finalmente, mostramos por pantalla un mensaje indicando que ha finalizado la ejecución del programa principal.

Si lanzamos ahora el script, el resultado es el siguiente:

```
Fin de la ejecución  
Hola Prioritario 1  
Hola Prioritario 2  
Hola 4  
Hola 3  
Hola 2  
Hola 1  
Hola 0
```

Si nos damos cuenta, da la sensación que las órdenes se han ejecutado en orden prácticamente inverso al que esperaríamos en la programación secuencial tradicional. Vamos a ver el por qué de todo esto.

1.1 El event Loop o bucle de eventos

Nodejs gestiona la concurrencia mediante un único hilo de ejecución, de forma que lo primero que se ejecuta es el programa principal. Si durante la ejecución de este programa principal se producen ciertos eventos, estos se añaden a una cola de eventos.

En este momento, entra en acción el *Event Loop*, que es un bucle que está siempre en ejecución, y que tiene la misión de vigilar la cola de eventos e invocar los *callbacks* correspondientes. Cuando un callback finaliza, retorna el control al bucle de eventos para que siga gestionando el resto de cola.

Si durante la ejecución se produce otro evento, dicho evento se añade a la cola sin interrumpir la ejecución actual. De este modo nos aseguramos que una función que se lanza como respuesta a un evento empieza y acaba, evitándonos así accesos concurrentes a secciones críticas y otros problemas de concurrencia de la programación secuencial. Cuando la cola de eventos se encuentre vacía, se devuelve el control al sistema operativo.



Disponéis de más información sobre el bucle de eventos en el artículo Javascript Asíncrono de LemonCode.

2 Promesas en ES6

Las *Promises* o *Promesas* son objetos que representan el estado de finalización de una operación asíncrona. A pesar de que podemos crear promesas mediante su constructor, lo más habitual será ser “consumidores” de promesas que nos devuelven las funciones asíncronas.

Las promesas se utilizan como alternativa a las funciones de callback, de forma que estas funciones de callback sean funciones que se ejecutan como métodos de los objetos *Promesa*.

2.1 Ejemplo

Vemos un ejemplo de creación y uso de promesas. Partimos del siguiente código con funciones de callback:

```
function divide(dividendo, divisor, cbOk, cbErr) {
  if (divisor == 0) cbErr("División por cero!");
  else cbOk(dividendo / divisor);
}

let a = 10;
let b = 0;

divide(
  a,
  b,
  function(result) {
    // Callback si la función tiene éxito
    console.log("El resultado de " + a + "/" + b + " es " + result);
  },
```

```
function(error) {  
    // Callback si la función falla  
    console.log("Error: " + error);  
};
```

como vemos, tenemos la función `divide` que comprueba primeramente que el divisor no sea cero. Si el divisor es 0, invoca la función de callback `cbErr`, lanzando un mensaje de excepción. En caso de que el divisor no sea cero, invocará la primera función de callback `CbOk` pasándole el resultado.

De este modo, cuando invocamos la función `divide`, le pasamos como argumentos el dividendo, el divisor, así como una función de callback a ejecutar si todo es correcto, y otra para cuando aparezcan errores.

Para realizar esta función con promesas, haríamos lo siguiente:

```
function divide(dividendo, divisor) {  
    return new Promise((resolve, reject) => {  
        if (divisor == 0) reject("División por cero!");  
        else resolve(dividendo / divisor);  
    })  
}  
  
let a = 10;  
let b = 0;  
  
divide(a, b)  
    .then(result => {  
        console.log("El resultado de " + a + "/" + b + " es " + result);  
    })  
    .catch(error => {  
        // Callback si la función falla  
        console.log("Error: " + error);  
    })
```

Analicemos el código:

```
function divide(dividendo, divisor) {  
    return new Promise((resolve, reject) => {  
        if (divisor == 0) reject("División por cero!");  
        else resolve(dividendo / divisor);  
    })  
}
```

Como vemos, ahora la función `divide` recibe únicamente el dividendo y el divisor, sin ninguna función de callback. En lugar de ello, lo que hace es devolver un objeto de tipo `Promise`. Al constructor de esta clase le tenemos que pasar una función anónima (en forma de *función flecha* en el ejemplo), que recibirá dos funciones más: `resolve` y `reject`. La primera se usará para devolver información en caso de éxito y la segunda para informar de alguna excepción.

Vemos ahora la parte interesante, en la que hacemos de *consumidores* de estas promesas:

```
divide(a, b)
  .then(result => {
    console.log("El resultado de " + a + "/" + b + " es " + result);
  })
  .catch(error => {
    // Callback si la función falla
    console.log("Error: " + error);
  })
```

En el ejemplo hemos invocado directamente la función `divide`, pasándole solo el dividendo y el divisor. Como hemos comentado, esta función nos devolvía un objeto de tipo `Promise`. Pues bien, este objeto dispone de dos métodos: `then` y `catch`, que sirven para indicar qué queremos hacer en caso de que la promesa se resuelva exitosamente, en el caso de `then`, o qué queremos hacer si esta es rechazada, en el caso del `catch`. Como vemos, esto se indica dentro de cada método con una función.



Resolución de una promesa

Un objeto de tipo promesa posee tres estados:

- Pendiente (*pending*),
- Resuelta (*fulfilled*), o
- Rechazada (*rejected*).

Cuando se crea una promesa dentro de una función asíncrona, esta se encuentra en estado *Pendiente*. Cuando la función tiene preparada la respuesta (lo que sería un `return`) en una función síncrona, si esta no ha producido ninguna excepción, se invoca al método `resolve` de la promesa, pasando esta a estado *resuelta*. **Decimos que en este momento, se resuelve la promesa** (esta nomenclatura la utilizaremos de forma frecuente a lo largo del texto). En caso de que se haya producido alguna excepción, se invocará al método `reject`, de forma que la promesa se *rechaza* (pasa en estado *rechazado*).

2.2 Ventajas con el uso de promesas

Cómo vemos, se trata de realizar llamadas asíncronas de una forma diferente a cómo utilizábamos los callbacks, que presenta ciertas ventajas:

2.2.1 Garantías

Las promesas nos garantizan:

- Que las funciones de callback no se invocarán antes de que acabe la ejecución actual del bucle de eventos de JavaScript,
- Que las funciones de callback añadidas con `.then`, se invocarán después de que la promesa se resuelva, con éxito o fracaso,
- Que se pueden añadir múltiples funciones de callback invocando a `.then` varias veces, para ser ejecutadas en la orden de inserción.

2.2.2 Encadenamiento

El encadenamiento nos permite ejecutar dos operaciones asíncronas seguidas y en orden. Dado que el resultado de la ejecución de `.then` es otra promesa, podemos utilizar diversos `.then` encadenados (cadena de promesas), de forma que los resultados (o la promesa resultante) de uno se pasan como entrada al siguiente. Aquí es donde vemos el verdadero potencial de las promesas: la función `.then`, nos devuelve una promesa nueva, diferente a la original, de forma que ofrece un nuevo método `.then` para tratar los resultados. Con el encadenamiento evitamos pues el temido *callback hell* o *pyramid of doom*.

2.3 Ejemplo

Veamos el siguiente código JS que utiliza callbacks para rellenar un vector con una lista de la compra:

```
const llistaCompra = []

function crearLlista() {

  setTimeout(function() {
    llistaCompra.push('Arros');
  });
}
```



```
    setTimeout(function() {
        llistaCompra.push('Bajoqueta');

        setTimeout(function() {
            llistaCompra.push('Garrofó');

            setTimeout(function() {
                llistaCompra.push('Pollastre');
            }, 1000);
        }, 1000);
    }, 1000);
}

crearLlista();
console.log(llibraCompra.toString()); // Eixida : []
```

Si ejecutamos el código anterior, observaremos que éste no muestra nada. La causa se debe al propio funcionamiento del bucle de eventos. Analicemos el código.

El código principal se compone de una invocación a `crearLlista()`, y el `console.log` del resultado. Dentro de `crearLlista`, lo que se hace es establecer un temporizador, que al cabo de 1000 ms (un segundo), dispara una función que añade el primer elemento a la lista:

```
function crearLlista() {

    setTimeout(function() {
        llistaCompra.push('Arros');
        ...
    }, 1000);
}
```

Dado que este evento se añade a la cola, registrando también el correspondiente callback, el código del programa principal, en el que se incluye el `console.log` ya se ha ejecutado cuando se añade el elemento al vector.

Para resolver esto, examinamos el flujo en el que se ejecutarán las diferentes funciones, y nos aseguramos que el procesamiento final (el `console.log`) se ejecuta al final.

El siguiente código resuelve pues el problema:

```
const llistaCompra = []

function crearLlista() {

  setTimeout(function () {
    llistaCompra.push('Arros');

    setTimeout(function () {
      llistaCompra.push('Bajoqueta');

      setTimeout(function () {
        llistaCompra.push('Garrofó');

        setTimeout(function () {
          llistaCompra.push('Pollastre');
          console.log(llibraCompra.toString());
          // Eixida : [Arros,Bajoqueta,Garrofó,Pollastre]

        }, 1000);
      }, 1000);
    }, 1000);
  }, 1000);
}

crearLlista();
```

No obstante, como vemos, nos encontramos ante un encadenamiento de llamadas, conocido como *Callback Hell*, que como vemos, suele dificultar bastante la lectura y comprensión del código.

Una mejor solución la tenemos con el uso de **Promesas**. Veamos cómo adaptar el código anterior para utilizar éstas. Mostramos el código comentado con las explicaciones:

```
const llistaCompra = []

// En primer lugar, creamos una función que añade un ítem
// proporcionado como argumento a la lista de la compra.
// Para ello, establece también el temporizador, para que
// esto se haga al cabo de un segundo, pero ahora
// lo que devolvemos es una promesa.
//
// La resolución de esta promesa consistirá pues en
// añadir el ítem proporcionado a la lista.
```

```
function afigItem(item) {
    return new Promise(resolve => setTimeout(function() {
        resolve(llistaCompra.push(item));
    }, 1000));
}

// En segundo lugar, Vamos añadiendo los diferentes
// componentes de la lista mediante la función anterior.
// Cada invocación a esta función devuelve una promesa,
// cuyo resultado tratamos con la función then.
// Dado que en cada then añadimos otra invocación, podemos
// encadenar todas estas promesas del siguiente modo.

afigItem("Arros")
    .then(function() {
        return afigItem("Bajoqueta");
    })
    .then(function() {
        return afigItem("Garrofó");
    })
    .then(function() {
        return afigItem("Pollastre");
    })
    .then(function() {
        console.log(llistaCompra.toString());
    })
    .then(function() {
        // ...
    });
```

2.3.1 async/await

La versión EcmaScript 2017 introdujo la mejora sintáctica *async/await*, que permite trabajar con funciones asíncronas como si trabajásemos con funciones síncronas.

Hay que remarcar que se trata de una mejora sintáctica, construida sobre promesas, de forma que las funciones que retornaban promesas son exactamente iguales que antes.

Así pues, siguiendo con el ejemplo anterior, en primer lugar, dejamos la función `afigItem` tal y como la teníamos.

```
function afigItem(item) {
```

```
return new Promise(resolve => setTimeout(function() {  
    resolve(llistaCompra.push(item));  
}, 1000));  
}
```

Posteriormente, definimos una función `crearLlista` que invocará la función `afigItem` tantas veces como items haya que incorporar, proporcionándole éstos.

La diferencia es que ahora, precederemos cada invocación la palabra clave `await`, de modo que esperemos a que finalice la ejecución antes de pasar a la siguiente línea. Al utilizar `await` vamos a tener que definir esta función con el método `async`, que simplemente indica que se va a usar `await` dentro de la función:

```
async function crearLlista() {  
    await afigItem("Arros");  
    await afigItem("Bajoqueta");  
    await afigItem("Garrofó");  
    await afigItem("Pollastre");  
    console.log(llistaCompra.toString());  
}  
  
crearLlista();
```